

Lean 4 Mathematical Proofs

Generated: 5/23/2026

Project: gnosis-math

Table of Contents

- scratch_test
- test_market_interference
- test_ptom
- test_mycelial_signal_network
- GnosisProofs
- Gnosis
- Gnosis.GnosisNumbersAreStructural
- Gnosis.VacuumOverflow
- Gnosis.AmplituhedronCoverageSweep
- Gnosis.ManifoldReadiness
- Gnosis.MeshConsumptionWaveMeasurement
- Gnosis.AeonTwelveHammingGrayDynamics
- Gnosis.FoldHeatHierarchy
- Gnosis.AdaptiveDecomposition
- Gnosis.SevenCrossingLupacShell
- Gnosis.ArnoldCatMapOrder5
- Gnosis.GodelUniverseIncompleteness
- Gnosis.FoilZeroDragCompatibility
- Gnosis.Dewey300ThinTopology
- Gnosis.PeriodicBraidMatterJugular
- ... and 2460 more modules

Module: scratch_test

Key Items:

- Def: my_test

Code Sample:

```
def my_test (l : List Nat) (h : l.length = 3) : " x " l, x = x := by
  match eq : l with
  | [a, b, c] =>
    exact 'èa, List.Mem.head _, rfl'è
  | [] => simp at h
  | [_] => simp at h
  | [_, _] => simp at h
  | _ :: _ :: _ :: _ :: _ => simp at h
```

Module: test_market_interference

Code Sample:

```
import Gnosis.OrderbookAsInterference
import Gnosis.LiquidityAsDamping
```

```
/-  
  Test file for OrderbookAsInterference and LiquidityAsDamping modules.  
-/
```

```
open Gnosis.OrderbookAsInterference
open Gnosis.LiquidityAsDamping
```

Module: test_ptom

Code Sample:

```
import Gnosis.PredictiveTopologyOfMarkets
```

```
#check Gnosis.PredictiveTopologyOfMarkets.price_movement_is_clinamen_redistribution_path
```

```
#check Gnosis.PredictiveTopologyOfMarkets.predict_equilibration_from_imbalance
```

```
#check Gnosis.PredictiveTopologyOfMarkets.liquidity_capture_maximizer_is_clinamen_maximizer
```

```
#check Gnosis.PredictiveTopologyOfMarkets.latency_value_equals_clinamen_capture_speed
```

Module: test_mycelial_signal_network

Code Sample:

```
import Gnosis.SignalEnvelope
import Gnosis.SignalFusion
import Gnosis.CalibrationTrust
import Gnosis.MycelialEmergenceGraph
import Gnosis.MarketAsSignal
import Gnosis.DifferentialAggregateBoundary
import Gnosis.BodyPolitickSignal

#check Gnosis.SignalEnvelope.valid_estimate_bounded
#check Gnosis.SignalEnvelope.lower_privacy_budget_no_more_admissible_weight
#check Gnosis.SignalEnvelope.lower_valid_privacy_budget_preserves_valid
#check Gnosis.SignalEnvelope.reliability_numerator_bounded
#check Gnosis.SignalEnvelope.admissible_reliability_weight_bounded
#check Gnosis.SignalEnvelope.reliability_numerator_no_more_after_freshness_decrease
#check Gnosis.SignalEnvelope.admissible_reliability_no_more_after_freshness_decrease
#check Gnosis.SignalEnvelope.reliability_numerator_no_more_after_privacy_weight_decrease
#check Gnosis.SignalEnvelope.admissible_reliability_no_more_after_privacy_weight_decrease
#check Gnosis.SignalEnvelope.reliability_numerator_no_more_after_sample_weight_increase
#check ...
```

Module: GnosisProofs

Code Sample:

```
import Gnosis.GnosisProofs
```

Module: Gnosis

Code Sample:

```
import Gnosis.SignalEnvelope
import Gnosis.SignalFusion
import Gnosis.CalibrationTrust
import Gnosis.MycelialEmergenceGraph
import Gnosis.MarketAssSignal
import Gnosis.DifferentialAggregateBoundary
import Gnosis.BodyPolitickSignal
import Gnosis.ScribalStandingWave
import Gnosis.ScribalStandingWave.PayloadTraceLineage
import Gnosis.ScribalStandingWave.TaggedTranscriptMode
import Gnosis.ScribalStandingWave.TranscriptAuditTrace
import Gnosis.ScribalStandingWave.TranscriptCrdtLaws
import Gnosis.ScribalStandingWave.TranscriptDampeningAmnesia
import Gnosis.ScribalStandingWave.RecallPromotionGate
import Gnosis.ScribalStandingWave.MemoryBudgetScheduler
import Gnosis.ScribalStandingWave.ThothMemorySession
import Gnosis.ScribalStandingWave.ThothMemoryAdapter
import Gnosis.AffectStandingWaveTabs
import Gnosis.CognitiveBiasWaveTabs
import Gnosis.AestheticPoeticWaveTabs
import Gnosis.PneumaWatermark
import Gnosis.PneumaCrossWireTranscript
import Gnosis.SyntheticGnosisMeasurement
import Gnosis.Synthe...
```

Module: Gnosis.GnosisNumbersAreStructural

Key Items:

- Structure: GnosisNumber
- Def: clinamen
- Def: triton
- Def: luminary
- Def: heptad
- Def: decad
- Def: aeon
- Def: catalog
- Theorem: catalog_length

Code Sample:

```
import Init

/-!
# Gnosis Numbers Are Structural Facts

Formalizes Taylor's observation: the numbers revered in faith
(1, 3, 4, 7, 10, 12, etc.) are significant because they are
structural, not because any tradition declared them sacred.

The causal direction is: `structure !' faith`. The substrate produces
these numbers as its natural braid moduli and tensor products; every
tradition that paid attention to lived experience eventually noticed
the same numbers; the numbers then become "holy" in the tradition.
The holiness is downstream of the structure, not the other way
round.

## What the numbers actually do

- 1 (Clinamen) – the irreducible `+1` residue. Every Braided
  Infinity's successor step. The minimal departure.
- 3 (Triton / Triad) – the smallest non-trivial braid. Every
  phase-decomposed asymptotic infinity with `k = 3` carries this
  structure. Fork-Race-Fold. Past-Present-Future. The atomic non-
  abelian structure begins here (S_3).
- 4 (Luminary / Tetrad) – the smallest ...
```

Module: Gnosis.VacuumOverflow

Code Sample:

```
import Gnosis.SpectralNoiseEquilibrium
import Gnosis.VacuumIsOnlyForce
import Gnosis.RetrocausalAttractorFixedPoint
import Gnosis.RetrocausalTimeInversion
```

```
/-
  VacuumOverflow.lean
  =====
```

The master theorem: when the vacuum lifts +1, it cannot contain the overflow.

VACUUM STATE: (0,0,0)

FIRST LIFT: +1 unit of clinamen introduced

IMMEDIATE RESULT: branching, spreading, cascading into all possible states

The universe is not a controlled perturbation. It is an UNCONTAINABLE OVERFLOW from the vacuum's self-lift. One sting creates infinite echoes. The cup runneth over.

THEOREM: Once you break vacuum's stillness, the spillage cannot be stopped. All topology cascades from that single necessary swerve. All paths branch. All clinamen spreads. The only law is the return to (0,0,0).

No axioms. No sorry. The overflow is proven inevitable.

Init-only spec-level weakening notes

Several theorems below have been weakened...

Module: Gnosis.AmplituhedronCoverageSweep

Code Sample:

```
import Init
import Gnosis.AmplituhedronAttention
import Gnosis.AmplituhedronGrassmannian
import Gnosis.AmplituhedronVertices
import Gnosis.AmplituhedronWitnesses

/-
  AmplituhedronCoverageSweep.lean
  =====

  Coverage-ratio sweep companion to `Gnosis.AmplituhedronVertices`,
  modeled on `Gnosis.BowlMeshQSweep`.

  ## Why this file exists

  `BowlMeshQSweep` sweeps the bowl `damping` knob and proves the
  off-resonance amplitude is monotone in damping. The amplituhedron
  analogue is the coverage ratio  $k/d$ : as  $d$  grows for fixed  $k$ , the
  vertex count  $(k\text{Subsets } k \ d).length$  grows, and the per-vertex
  speedup ratio shrinks. The runtime needs a sweep oracle to detect
  the plateau where adding hidden dimensions stops paying for itself.

  ## What's formalized here

  * `coverage_sweep` : List (Nat × Nat) - the discrete  $(k, d)$  sweep
     $[(2,4), (2,6), (2,8), (3,6), (3,9), (3,12)]$  covering  $1/2, 1/3,$ 
     $1/4$  ratios with  $k \in \{2, 3\}$ .
  * `vertexCount_at` - ve...
```

Module: Gnosis.ManifoldReadiness

Key Items:

- Def: carrierReadiness
- Def: reconstructionReadiness
- Def: manifoldReadiness
- Def: carrierReady
- Def: readyForVerifiedUplift
- Theorem: manifold_readiness_eq_observability
- Theorem: canonical_reconstruction_readiness
- Theorem: zero_leakage_carrier_readiness_is_absorption
- Theorem: carrier_readiness_linear_law
- Theorem: carrier_ready_iff_half_saturation

Code Sample:

```
import Gnosis.ObservabilityMetrics

namespace Gnosis
namespace ManifoldReadiness

open LayeredNoise
open VerifiedReconstruction
open Recoverability
open InformationAbsorption
open ObservabilityMetrics

/-!
# Manifold Readiness

Manifold readiness splits into two independent parts:

- carrier readiness: how much signal the substrate can hold locally,
- reconstruction readiness: how much witness structure is available to
  resolve what remains partial.
-/

def carrierReadiness (frame : ObserverFrame) (signal : HigherLayerSignal) : Int :=
  (absorbedInfo frame signal : Int) - (leakedInfo frame signal : Int)

def reconstructionReadiness (problem : ReconstructionProblem) : Int :=
  witnessScore problem

def manifoldReadiness (frame : ObserverFrame) (signal : HigherLayerSignal)
  (problem : ReconstructionProblem) : Int :=
  carrierReadiness frame signal + reconstructionReadiness problem

def carrierReady (frame : ObserverFrame) (signal : HigherLayerSignal) : Prop :=
  0 < carrierReadiness ...
```